

Scope and Call Frames

Lecture 11

CS 1001

Kirk Duran

Today

- ▶ Define **local**, **enclosing**, and **global** scopes precisely.
- ▶ Separate lexical name resolution from runtime call-frame organization.
- ▶ Trace nested functions and identify free-variable bindings.
- ▶ Use `global` and `nonlocal` to change where assignment binds a name.
- ▶ Bind arguments using position, keywords, and default parameter values.
- ▶ Diagnose common scope and argument-binding errors by tracing program state.

Key idea

A name has meaning only relative to a scope, while a running function call has state only relative to its call frame.

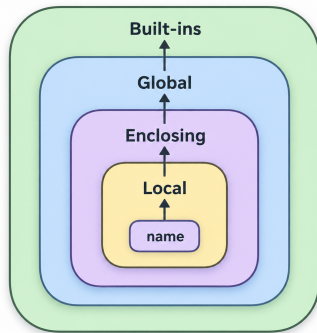
Part 1: Scope and Name Resolution

Names Are Resolved Within Scopes

Definition

A **scope** is a region of program text in which a binding for a name is visible.

- ▶ Assignment, parameters, imports, and function definitions can bind names.
- ▶ Using a name requires Python to determine which visible binding the occurrence refers to.
- ▶ The nearest applicable lexical binding is selected.



Local and Global Scope

Definition

A **local binding** is a name bound within a function body and belonging to that function's local scope.

Example

```
def f(x):  
    y = x + 1
```

Here `x` and `y` are local to `f`.

Definition

A **global binding** is a name bound at module level.

Example

```
tax_rate = 0.075  
  
def total(price):    return price *  
    (1 + tax_rate)
```

A Function May Read a Global Binding Without `global`

Program

```
rate = 2  
  
def scale(value):  
    return value * rate  
  
answer = scale(5)
```

1. `value` is found in the local scope of `scale`.
2. `rate` has no local binding in `scale`.
3. Name resolution continues outward and finds global `rate` -> 2.
4. The function returns 10.

Key idea

Reading a global name and rebinding a global name are different operations.

Assignment Inside a Function Normally Creates a Local Binding

Program

```
count = 10
def reset():
    count = 0
    return count
x = reset()
```

Global binding

count -> 10

The module-level count remains 10.

During reset

local count -> 0
return 0

Shadowing Creates a Different Binding with the Same Name

Definition

Shadowing occurs when a nearer scope binds a name that also exists in an outer scope.

Example

```
price = 100
def discounted():
    price = 80
    return price
```

- ▶ Inside `discounted`, `price` resolves to the local binding 80.
- ▶ Outside the function, `price` still resolves to the global binding 100.
- ▶ Same spelling does not imply same binding.

A Subtle Rule: Binding Anywhere Makes the Name Local

Program

```
x = 10
def f():
    print(x)
    x = 20
```

Result

Calling `f()` raises `UnboundLocalError`.

- ▶ Because `x` is assigned somewhere in `f`, Python treats `x` as local throughout that function block.
- ▶ The first `print(x)` therefore attempts to read the local `x` before that local binding has received a value.

Key idea

Scope classification is not determined statement-by-statement during execution.

Part 2: Call Frames as Runtime State

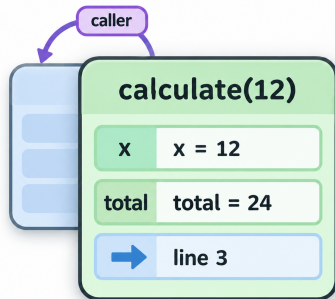
A Call Frame Represents One Active Execution of a Function

Definition

A **call frame** is the runtime execution state associated with one active function call.

A useful CS 1001 model records:

- ▶ parameter bindings;
- ▶ local-variable bindings;
- ▶ the currently executing location;
- ▶ the relationship to the caller.



Frame Lifetime Follows Call Lifetime

1. Evaluate the arguments at the call site.
2. Begin the call and initialize the frame's parameter bindings.
3. Execute the function body, adding or changing local bindings.
4. Evaluate a `return` expression or reach the end of the function.
5. End the active call; the caller resumes using the produced value.

During call

frame exists
locals are inspectable

After return

call is no longer active
its ordinary local environment is no longer active

Two Calls to the Same Function Use Different Frames

Program

```
def increase(value):  
    result = value + 1  
    return result  
  
a = increase(4)  
b = increase(10)
```

First call

```
value -> 4  
result -> 5
```

Second call

```
value -> 10  
result -> 11
```

The parameter name `value` is reused, but the two runtime bindings belong to different calls.

Call Stack and Scope Answer Different Questions

Call stack

Runtime structure.

Question: Which calls are active, and which caller resumes next?

Lexical scope

Program-text structure.

Question: Which binding does this occurrence of a name refer to?

Important

A caller's local variables do not automatically become visible to a callee merely because the caller's frame is beneath it on the call stack.

Trace a Call Frame, Not Just the Final Result

Program

```
def compute(a, b):  
    total = a + b  
    scaled = total * 2  
    return scaled  
  
answer = compute(3, 5)
```

Execution point	Local frame bindings
function entry	a -> 3, b -> 5
after total = a + b	a -> 3, b -> 5, total -> 8
after scaled = total * 2	add scaled -> 16
return	produce 16

Part 3: Nested Functions and Enclosing Scope

A Function Definition May Appear Inside Another Function

Example

```
def outer(value):  
    def double(x):  
        return x * 2  
    return double(value)
```

- ▶ Executing the inner `def` statement creates a function object and binds it to the local name `double`.
- ▶ That binding belongs to the current call of `outer`.
- ▶ The inner function can be called like any other function value while its binding is available.

Nested Functions Add an Enclosing Function Scope

Program

```
def outer(base):  
    offset = 3  
    def inner(x):  
        return x + offset  
    return inner(base)
```

- ▶ `x` is local to `inner`.
- ▶ `offset` is not local to `inner`; it is a **free variable** there.
- ▶ Python resolves `offset` in the nearest enclosing function scope where it is bound.

Key idea

Lexical nesting, not the order of callers on the stack, determines enclosing-scope name resolution.

Trace a Nested Function Call

Call

```
result = outer(5)
```

outer frame

```
base -> 5  
offset -> 3  
inner -> <function>
```

Then outer returns 8, so result -> 8.

inner frame

```
x -> 5  
free offset -> 3 from enclosing scope  
return 8
```

nonlocal Rebinds the Nearest Enclosing Function Binding

Program

```
def outer():  
    count = 0  
    def step():  
        nonlocal count  
        count = count + 1  
    step()  
    return count
```

- ▶ `nonlocal count` prevents assignment from creating a new local `count` in `step`.
- ▶ The assignment targets the existing `count` in the nearest enclosing function scope.
- ▶ `outer()` therefore returns 1.

Part 4: The global Statement

global Changes the Binding Target of a Name

Program

```
count = 0
def increment():
    global count
    count = count + 1
```

Definition

Within the current scope, `global name` declares that uses and assignments of `name` refer to the module-level global binding rather than to a new local binding.

Key idea

`global` is a lexical declaration about name resolution and assignment target selection; it does not copy a global value into the function. It must precede uses of that name in the same function block.

Trace a Global Rebinding

Program

```
score = 10  
  
def add_points(points):  
    global score  
    score = score + points  
    return score  
  
result = add_points(5)
```

1. Frame begins with local parameter `points` -> 5.
2. `score` resolves to global 10 because of the declaration.
3. Assignment rebinds global `score` -> 15.
4. The call returns 15; then `result` -> 15 globally.

Without `global`, the Same Code Is Not Equivalent

Problematic version

```
score = 10

def add_points(points):
    score = score + points
    return score
```

Why it fails

The assignment makes `score` local to `add_points`. The right-hand side then tries to read that local `score` before it has been bound, producing `UnboundLocalError`.

Key idea

Adding an assignment can change how every occurrence of that name is classified within the function block.

Local, nonlocal, and global Select Different Binding Targets

Inside a function	Assignment target	Typical meaning
ordinary assignment	current local scope	create/rebind local name
<code>nonlocal x</code>	nearest enclosing function binding	rebind enclosing state
<code>global x</code>	module-level global binding	rebind global state

Key idea

These declarations alter where a name is resolved for assignment; they do not change the right-hand-side evaluation rules. Prefer explicit parameters and return values when shared mutable state is unnecessary.

Part 5: Parameter Binding Beyond Position

Positional Arguments Bind by Parameter Position

Definition and call

```
def rectangle(width, height):    return width * height  
area = rectangle(5, 3)
```

Binding

```
first argument 5 -> width  
second argument 3 -> height
```

Position determines the correspondence; the caller's variable names, if any, are irrelevant after their values are evaluated.

Keyword Arguments Bind by Parameter Name

Calls

```
rectangle(width=5, height=3)  
rectangle(height=3, width=5)
```

- ▶ A keyword argument explicitly identifies the parameter slot to fill.
- ▶ Keyword argument order therefore does not determine parameter correspondence.
- ▶ The argument expression is still evaluated to obtain a value before the call executes.

Key idea

Positional binding answers “which slot by order?”; keyword binding answers “which slot by parameter name?”

Positional and Keyword Arguments May Be Mixed

Definition

```
def describe(name, age, city):    return name, age, city
```

Valid call

```
describe("Ada", age=20, city="SLO")
```

1. Positional argument "Ada" fills name.
2. Keyword age=20 fills age.
3. Keyword city="SLO" fills city.

In ordinary calls, explicit positional arguments precede explicit keyword arguments.

Binding must be unambiguous

A parameter may not receive two values, and every required parameter must be filled. Duplicate, missing, or unknown arguments raise `TypeError`.

Default Parameter Values Fill Omitted Slots

Definition

```
def power(base, exponent=2):    return base ** exponent
```

`power(5)`

base -> 5

exponent -> 2 from default

return 25

`power(5, exponent=3)`

base -> 5

exponent -> 3 from argument

return 125

Default Expressions Are Evaluated When the Function Is Defined

Trace carefully

```
x = 10
def show(value=x):
    return value
x = 20
answer = show()
```

1. When the def executes, the default expression x evaluates to 10.
2. Rebinding global x -> 20 does not recompute the stored default.
3. show() therefore binds value -> 10.

Design consequence

Mutable objects used as defaults are shared across calls that omit that argument; this is usually avoided in introductory code.

Python Supports Multiple Parameter Kinds

General form

```
def f(a, b, /, c, d=0, *, scale=1): ...
```

Parameters	How arguments may be supplied
a, b before /	positional only
c, d	positional or keyword
scale after *	keyword only

Key idea

Most student-defined functions use positional-or-keyword parameters; / and * make an interface more explicit when needed.

Part 6: Integrated Tracing and Debugging

Integrated Trace: Global and Local Bindings with Keywords

Program

```
rate = 2
def transform(value, offset=1):
    scaled = value * rate
    return scaled + offset
answer = transform(offset=4, value=3)
```

1. Keyword binding creates local value $\rightarrow$ 3, offset $\rightarrow$ 4.
2. Global lookup supplies rate $\rightarrow$ 2.
3. Local assignment creates scaled $\rightarrow$ 6.
4. Return expression evaluates $6 + 4 \rightarrow 10$.
5. The caller binds global answer $\rightarrow 10$.

Integrated Trace: Nested Function and Enclosing Binding

Program

```
def make_total(base, tax=0.10):  
    def add_tax(amount):  
        return amount * (1 + tax)  
    return add_tax(base)  
  
result = make_total(50, tax=0.20)
```

Outer frame

```
base -> 50  
tax -> 0.20  
add_tax -> <function>
```

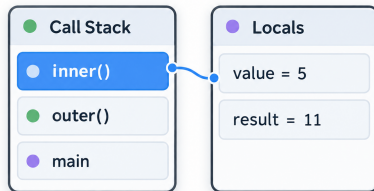
Inner frame

```
amount -> 50  
free tax -> 0.20  
return 60.0
```

Debugger Strategy: Inspect the Correct Frame

When stopped inside nested calls:

1. Identify the currently executing frame.
2. Inspect its parameter and local bindings.
3. Use the call stack to select an outer active frame when needed.
4. Keep lexical scope separate from the debugger's stack ordering.
5. Predict name resolution before relying on the debugger display.



Tracing Activity: Resolve Every Name Before Computing

Predict without running

```
x = 5
def outer(x, scale=2):
    offset = 3
    def inner(value):
        return value * scale + offset
    return inner(x)
answer = outer(scale=4, x=2)
```

In class

Draw the active frames during `inner`. For each occurrence of `x`, `scale`, `offset`, and `value`, identify its binding and scope. Then determine `answer`.

Takeaways

- ▶ **Scope** determines which binding a name occurrence can refer to; a **call frame** records runtime state for one active call.
- ▶ Ordinary assignment inside a function creates or rebinds a local name unless `nonlocal` or `global` changes the binding target.
- ▶ Nested functions may read names from lexically enclosing function scopes.
- ▶ `global` permits rebinding module-level names, but it also introduces externally visible state changes that must be traced explicitly.
- ▶ Function-call argument binding fills parameter slots using positional arguments, keyword arguments, and default values.
- ▶ Accurate tracing separates three questions: *Which frame is active? Which scope supplies this name? Which value is bound?*

Key idea

The central discipline is to locate the binding first, then evaluate the expression.